

Geniřletilmiř Gazebo Sentez Raporu:

Kapsamlı Mimari Analizi ve XML Yapılandırma Kılavuzu

1. Giriř: Gazebo Ekosisteminin Katmanları

Bu rapor, Gazebo simülasyon ekosisteminin (Gazebo Classic 11.15.1 ve SDF/URDF standartları temel alınarak) derinlemesine, iki katmanlı bir analizini sunar. Başarılı bir simülasyon, bu iki katmanın da anlaşılmasını gerektirir:

- Mimari Katman (Perde Arkası - Nasıl Çalışır?):** Gazebo'nun temel felsefesi. Fizik ve sensör verisi üreten "sunucu" (`gzserver`) ile bunu görselleřtiren "istemci" (`gzclient`) arasındaki ayrım. Bu katman, C++ eklenti sistemi, çoklu fizik motoru desteęi ve dilden bağımsız iletişim (Gazebo Transport) gibi konuları içerir.
- Yapılandırma Katmanı (Kullanıcı Arayüzü - Nasıl Yapılandırılır?):** Bu mimariyi nasıl kontrol ettiğimiz. Bu katman, simülasyondaki her şeyi tanımlamak için kullanılan XML tabanlı dillerdir.

Bu raporda, bu iki katmanı bir araya getireceğiz. Teknik bir konsepti (örn. "Fizik Motoru Kararlılığı") açıkladıktan sonra, bu konsepti doğrudan kontrol eden XML düğümlerini (`<cfm>` , `<erp>`) sunacağız.

1.1 Temel İkilik: SDF vs. URDF

Ekosistemi anlamamanın anahtarı, iki XML formatı arasındaki farkı ve etkileřimi kavramaktır:

- SDF (Simulation Description Format):** Gazebo'nun yerel formatıdır. Bir simülasyon dünyasının tüm yönlerini tanımlayabilir: fizik, ışıklar, ortam, sensörler, eklentiler ve robotlar. `.world` veya `.sdf` dosyaları bu formatı kullanır.
- URDF (Unified Robot Description Format):** ROS'un (Robot Operating System) standart robot tanımlama formatıdır. Bir robotun *kinematik* ve *görsel* tanımına (linkler, eklemler, kütle) odaklanır. Ancak sensörleri, eklentileri, sürtünmeyi veya kapalı kinematik zincirleri yerel olarak **desteklemez**.

Gazebo, bir URDF dosyasını yüklerken onu **dahili olarak anında bir SDF modeline dönüřtürür**. Bu dönüřüm sırasındaki "bořlukları" (eksik sensörler, eklentiler) doldurmak için Bölüm 6'da detaylandırılacak olan özel `<gazebo>` etiketini kullanırız.

2. Temel Mimari ve Dünya Yapılandırması

Mimari Konsept (gzserver/gzclient): Gazebo'nun gücü, simülasyonun çekirdek işlevlerini (`gzserver`) görselleřtirmeden (`gzclient`) ayırmasından gelir. `gzserver` tüm "ağır iş yükünü" (fizik, sensör verisi) yapar ve grafik kartı olmayan sunucularda "arayüzsüz" (headless) çalışabilir. Bu iki süreç, Google Protocol Buffers (Protobufs) tabanlı bir iletişim katmanı (`Gazebo Transport`) üzerinden konuşur.

Bu mimarinin ana yapılandırma dosyası `.world` dosyasıdır ve simülasyonun "Tanrı" düzeyindeki parametrelerini içerir.

2.1 Fizik Motoru Ayarları `<physics>`

Mimari Konsept: Gazebo, Open Dynamics Engine (ODE), Bullet, Simbody ve DART gibi birden fazla "tak-çıkart" fizik motorunu destekler. Varsayılan ve en yaygın kullanılanı ODE'dir. Simülasyonun kararlılığı, hızı ve gerçekçilięi, doğrudan `.world` dosyasındaki `<physics>` etiketi altında ayarlanan parametrelere baęlıdır.

XML Yapılandırması: Aşağıdaki XML, ODE motoru için kritik performans ve kararlılık ayarlarını gösterir.

```
<physics type="ode" name="ode_default_profile">
  <!--
    Zaman Adımı: Bir simülasyon adımında ilerlenecek saniye.
    Doğruluk (düşük değer) vs. Performans (yüksek değer) değiş-tokuşu.
  -->
  <max_step_size>0.001</max_step_size>

  <!--
    Güncelleme Hızı: Simülasyonun saniyede kaç adım atmaya çalışacağı (Hz).
    (max_step_size * real_time_update_rate = 1.0) -> Gerçek zamanlı (1x) hedeflenir.
    Eğer 0 ayarlanırsa, donanımın izin verdiği en yüksek hızda (mümkün olduğunca hızlı) çalışır.
  -->
  <real_time_update_rate>1000</real_time_update_rate>

  <ode>
    <solver>
      <!-- 'quick' (hızlı, iteratif) veya 'world' (daha doğru, yavaş) -->
      <type>quick</type>
      <!--
        quick çözücünün iterasyon sayısı.
        Artırmak, eklem ve temas kararlılığını artırır ama yavaşlatır.
      -->
      <iters>70</iters>
      <sor>1.3</sor>
      <use_dynamic_moi_rescaling>false</use_dynamic_moi_rescaling>
    </solver>
    <constraints>
      <!--
        CFM (Constraint Force Mixing): KRİTİK PARAMETRE.
        Sert temasları "yumuşatarak" (bir yay gibi davranmasını sağlayarak)
        sistemi kararlı hale getirir.
        Simülasyon "patladığında" (jitter, fırlayan modeller) ayarlanacak ilk parametredir.
      -->
      <cfm>0.0</cfm>
      <!--
        ERP (Error Reduction Parameter): KRİTİK PARAMETRE.
        Her adımda eklem ve temas hatalarının (örn. penetrasyon) ne kadarının
        düzeltilileceğini belirler. CFM ile birlikte temasların "sertliğini" tanımlar.
      -->
      <erp>0.2</erp>
      <contact_max_correcting_vel>0.1</contact_max_correcting_vel>
      <contact_surface_layer>0.0001</contact_surface_layer>
    </constraints>
  </ode>
</physics>
```

2.2 Gelişmiş Arazi ve Çarpışma Özellikleri

Mimari Konsept: Simülasyon dünyası düz bir zeminden ibaret olmak zorunda değildir. Gazebo, Dijital Yükseklik Modellerini (DEM) kullanarak gerçek dünyadan alınmış karmaşık arazileri içe aktarabilir. Ayrıca, hangi nesnelerin birbiriyle çarpışıp çarpışmayacağını kontrol etmek için "bit maskeleme" adı verilen güçlü bir mekanizma sağlar.

XML Yapılandırması (DEM - Yükseltik Haritası):

```
<collision name="heightmap_collision">
  <geometry>
    <heightmap>
```

```

<!--
    GDAL kütüphanesi tarafından işlenebilen bir yükseklik haritası
    (örn. .dem, .tif).
-->
<uri>file://media/materials/textures/my_terrain.dem</uri>
<!--
    Haritanın simülasyondaki X, Y ve Z (yükseklik ölçeği)
    boyutları (metre).
-->
<size>129 129 10</size>
<!--
    DEM'in minimum yüksekliğini sıfırlamak için Z-ofseti
    ayarlamak amacıyla kullanılır.
-->
<pos>0 0 0</pos>
</heightmap>
</geometry>
</collision>

<!-- Görsel temsil için de aynı geometri kullanılır -->
<visual name="heightmap_visual">
    <geometry>
        <heightmap>
            <uri>file://media/materials/textures/my_terrain.dem</uri>
            <size>129 129 10</size>
            <pos>0 0 0</pos>
            <!-- Görsel için dokular eklenebilir -->
            <texture>
                <diffuse>file://media/materials/textures/dirt_diffusespecular.png</diffuse>
                <normal>file://media/materials/textures/flat_normal.png</normal>
                <size>1</size>
            </texture>
        </heightmap>
    </geometry>
</visual>

```

XML Yapılandırması (Çarpışma Bit Maskeleri - `<collide_bitmask>`):

Konsept: Her çarpışma geometrisine 16-bitlik bir sayı (maske) atanır. İki nesne (A ve B) karşılaştığında, bit düzeyinde AND işlemi uygulanır: `(A_mask & B_mask)` . Eğer sonuç `0` (sıfır) ise, çarpışma **yok** sayılır. Bu, performansı artırmak ve mantıksal ayırma (örn. robotun kendi parçalarıyla çarpışmasını engelleme) için kullanılır.

```

<!--
    ÖRNEK: Lidar (Grup 1) sadece Duvarlarla (Grup 2) çarpışsın,
    diğer Robotlarla (Grup 3) çarpışmasın.

    Lidar Maskesi: 0x01 (Grup 1)
    Duvar Maskesi: 0x02 (Grup 2)
    Robot Maskesi: 0x04 (Grup 3)
-->

<!-- Lidar'ın <collision> etiketi -->
<collision name="lidar_sensor_collision">
    <geometry>...</geometry>
    <!--
        Lidar'ın varsayılan maskesi (0xFFFF) yerine sadece Grup 2 (0x02)
        ve Grup 3 (0x04) ile çarpışmasını sağlayan bir maske verelim.
        Maske = 0x02 | 0x04 = 0x06
    -->
</filter>

```

```

    <category_bits>0x01</category_bits> <!-- Ben Grup 1'im -->
    <mask_bits>0x06</mask_bits> <!-- Sadece Grup 2 ve 3 ile çarpışırım -->
</filter>
</collision>

<!-- Duvarın <collision> etiketi -->
<collision name="wall_collision">
    <geometry>...</geometry>
    <filter>
        <category_bits>0x02</category_bits> <!-- Ben Grup 2'yim -->
        <mask_bits>0xFFFF</mask_bits> <!-- Her şeyle çarpışabilirim -->
    </filter>
</collision>

<!--
SONUÇ:
Lidar (0x01) vs Duvar (0x02):
Lidar Mask (0x06) & Duvar Category (0x02) = 0x02 (Çarpışma olur)
Duvar Mask (0xFFFF) & Lidar Category (0x01) = 0x01 (Çarpışma olur)

Lidar (0x01) vs Robot (0x04):
Lidar Mask (0x06) & Robot Category (0x04) = 0x04 (Çarpışma olur)
...ama biz olmasın istemiştik.
Doğru maske: Lidar sadece Duvarlarla (0x02) çarpışsın.
Lidar'ın mask_bits'i 0x02 olmalıdır.
-->

```

3. Robot Modellemenin Anatomisi: Linkler ve Eklemler

Mimari Konsept: Bir robot modeli, rijit gövdelerin (linkler) birbiriyle kısıtlı hareketlerle (eklemler) bağlanmasıyla oluşan bir koleksiyondur. Simülasyon optimizasyonu için temel bir prensip, bir linkin görsel temsilini (`<visual>`) ve onun **çarpışma** temsilini (`<collision>`) ayırmaktır.

- `<visual>` : Yüksek poligonlu, dokulu, estetik açıdan karmaşık bir 3D ağ (mesh). Render motoru (`gzclient`) tarafından kullanılır.
- `<collision>` : Fizik motoru (`gzserver`) tarafından hızlı hesaplama için kullanılan basit bir geometrik şekil (kutu, küre, silindir).

3.1 Linkler `<link>` : Fiziksel Varlık

```

<link name="chassis_link">
    <!--
        Statik/Dinamik: Bir modelin <static>true</static> olması,
        onun fizik motoru tarafından tamamen yok sayılmasını sağlar
        (örn. binalar, zemin). 'false' veya tanımsız olması dinamik olduğunu gösterir.
    -->
    <static>false</static>

    <!-- Linkin atalet özellikleri (fizik motoru için) -->
    <inertial>
        <mass>1.0</mass> <!-- Kütle (kg) -->
        <inertia> <!-- 3x3 dönme eylemsizlik matrisi -->
            <ixx>0.1</ixx> <ixy>0</ixy> <ixz>0</ixz>
            <iyy>0.1</iyy> <iyz>0</iyz>
            <izz>0.1</izz>
        </inertia>
    </inertial>

    <!-- Çarpışma modeli (fizik motoru için) - Basit geometri -->

```

```

<collision name="chassis_collision">
  <geometry>
    <box>
      <size>0.5 0.3 0.1</size>
    </box>
  </geometry>
  <!-- Bu linkin yüzey özellikleri (sürtünme vb.) -->
  <surface>
    <friction>
      <ode>
        <mu>1.0</mu> <!-- Birincil sürtünme katsayısı -->
        <mu2>1.0</mu2> <!-- İkincil sürtünme katsayısı -->

        <!--
          Torsional Sürtünme: Bir topun/tekerleğin dönmesini durduran
          gerçekçi dönme sürtünmesi. Varsayılan olarak kapalıdır
          (çünkü patch_radius 0'dır).
        -->
        <torsional>
          <coefficient>1.0</coefficient>
          <use_patch_radius>true</use_patch_radius>
          <patch_radius>0.01</patch_radius> <!-- Metre cinsinden temas yarıçapı -->
        </torsional>
      </ode>
    </friction>
    <contact>
      <ode>
        <max_contacts>10</max_contacts>
      </ode>
    </contact>
  </surface>
</collision>

<!-- Görsel model (render motoru için) - Karmaşık mesh -->
<visual name="chassis_visual">
  <geometry>
    <mesh>
      <uri>model://my_robot/meshes/chassis.dae</uri>
    </mesh>
  </geometry>
  <material>
    <script>Gazebo/Grey</script>
  </material>
</visual>
</link>

```

3.2 Eklemler <joint> : Linkleri Bağlama

Mimari Konsept: Bir <joint>, iki <link> 'i (bir parent ve bir child) birbirine bağlar ve aralarındaki hareketin türünü (örn. revolute - döner, prismatic - kayar, fixed - sabit) ve eksenini kısıtlar.

```

<joint name="left_wheel_joint" type="revolute">
  <!-- Kinematik zinciri tanımlar -->
  <parent>chassis_link</parent>
  <child>left_wheel_link</child>

  <!-- Eklemler konumu (ebeveayne göre) -->
  <pose>0 0.15 -0.05 0 0 0</pose>

  <!-- Hareketin gerçekleştiği eksen (burada Y eksenini etrafında döner) -->
  <axis>
    <xyz>0 1 0</xyz>
  </axis>

```

```

<limit>
  <!-- Döner eklemler için açısal limitler (radyan) -->
  <lower>-1e+16</lower> <!-- Sürekli dönüş için çok düşük/yüksek -->
  <upper>1e+16</upper>
</limit>
<dynamics>
  <!-- Eklem sürtünmesi (sönümleme) -->
  <damping>0.01</damping>
  <!-- Statik sürtünme (harekete başlamak için gereken tork) -->
  <friction>0.0</friction>
</dynamics>
</axis>

<!--
  Kuvvet/Tork Sensörü: Bu sensör bir <link>'e değil,
  bir <joint> etiketinin içine yerleştirilmelidir.
  Ekleme etkileyen kuvvetleri ve torkları ölçer.
-->
<sensor name="joint_force_torque" type="force_torque">
  <force_torque>
    <frame>child</frame> <!-- Ölçümlerin hangi çerçevede ifade edileceği -->
    <measure_direction>child_to_parent</measure_direction>
  </force_torque>
</sensor>
</joint>

```

4. Sensör Simülasyonu (Genişletilmiş)

Mimari Konsept: Sensörler (Kamera, Lidar, IMU), bir `<link>` 'e bağlı `<sensor>` etiketleri olarak SDF içinde tanımlanır. Gazebo, mükemmel ("ground truth") veriden, gerçekçi gürültü ve bozulmalara sahip verilere kadar bir "gerçekçilik spektrumu" sunar.

4.1 Lidar (Lazer) Sensörü `<ray>`

```

<sensor name="laser_sensor" type="ray">
  <always_on>true</always_on>
  <update_rate>10</update_rate>
  <visualize>true</visualize>
  <ray>
    <scan>
      <horizontal>
        <samples>640</samples> <!-- Yataydaki ışın sayısı -->
        <resolution>1</resolution>
        <min_angle>-2.26889</min_angle>
        <max_angle>2.26889</max_angle>
      </horizontal>
    </scan>
    <range>
      <min>0.08</min> <!-- Minimum algılama mesafesi -->
      <max>10.0</max> <!-- Maksimum algılama mesafesi -->
      <resolution>0.01</resolution> <!-- Mesafe çözünürlüğü -->
    </range>
    <noise>
      <!--
        Menzil (range) verisine Gaussian gürültü ekler.
        Bu, Lidar'ın "mükemmel" olmamasını sağlar.
      -->
      <type>gaussian</type>
      <mean>0.0</mean>
      <stddev>0.01</stddev> <!-- 1 cm standart sapma -->
    </noise>
  </ray>
</sensor>

```

```
</ray>
</sensor>
```

4.2 Kamera Sensörü <camera> ve Lens Bozulması <distortion>

Mimari Konsept: Kameralar, `gzclient` gibi OGRE render motorunu (`gzserver` içinde) kullanarak 2D görüntü (RGB) veya derinlik haritası (Depth Camera) verisi üretir. Gerçekçi lens bozulmalarını (SLAM/ Görsel Odometri testleri için kritik) simüle etmek için Brown's distorsiyon modelini destekler.

```
<sensor name="camera_sensor" type="camera">
  <update_rate>30.0</update_rate>
  <camera name="head_camera">
    <horizontal_fov>1.3962634</horizontal_fov>
    <image>
      <width>800</width>
      <height>800</height>
      <format>R8G8B8</format>
    </image>
    <clip>
      <near>0.02</near>
      <far>300</far>
    </clip>
    <noise>
      <!--
        GPU shader'ı ile çalışan, her pikselin her renk kanalına
        bağımsız olarak eklenen amplifikatör gürültüsü.
      -->
      <type>gaussian</type>
      <mean>0.0</mean>
      <stddev>0.007</stddev>
    </noise>
    <!--
      Lens Distorsiyonu: Gerçekçi kamera kalibrasyonunu
      simüle eder.
    -->
    <distortion>
      <k1>0.0</k1> <!-- Radyal bozulma katsayıları -->
      <k2>0.0</k2>
      <k3>0.0</k3>
      <p1>0.0</p1> <!-- Teğetsel bozulma katsayıları -->
      <p2>0.0</p2>
      <center>0.5 0.5</center> <!-- Görüntü merkezi -->
    </distortion>
  </camera>
</sensor>
```

4.3 IMU (Ataletsel Ölçüm Birimi) <imu>

Mimari Konsept: Gazebo'daki en karmaşık ve gerçekçi gürültü modellerinden birine sahiptir. Ham ivme ve açısal hız verilerine anlık Gaussian gürültü (`<noise>`) ve simülasyon boyunca sabit kalan "drift" hatasını simüle eden sapma/önyargı (`<bias>`) ekler.

```
<sensor name="imu_sensor" type="imu">
  <always_on>true</always_on>
  <update_rate>100</update_rate>
  <imu>
    <angular_velocity>
```

```

<x>
  <noise type="gaussian">
    <mean>0.0</mean>
    <!-- Anlık gürültünün standart sapması (rad/s) -->
    <stddev>2e-4</stddev>
    <!--
      "Drift" simülasyonu için sapmanın ortalaması.
      Simülasyon başında bir kez seçilir ve sabit kalır.
    -->
    <bias_mean>0.0000075</bias_mean>
    <!-- Sapmanın standart sapması (random walk) -->
    <bias_stddev>0.0000008</bias_stddev>
  </noise>
</x>
<y>...</y> <z>...</z> <!-- Y ve Z eksenleri için benzer bloklar -->
</angular_velocity>
<linear_acceleration>
  <x>
    <noise type="gaussian">
      <mean>0.0</mean>
      <!-- Anlık gürültünün standart sapması (m/s^2) -->
      <stddev>1.7e-2</stddev>
      <!-- Sapmanın ortalaması -->
      <bias_mean>0.1</bias_mean>
      <bias_stddev>0.001</bias_stddev>
    </noise>
  </x>
  <y>...</y> <z>...</z> <!-- Y ve Z eksenleri için benzer bloklar -->
</linear_acceleration>
</imu>
</sensor>

```

4.4 Mantıksal Kamera `<logical_camera>`

Mimari Konsept: Bu, algılama (perception) katmanını *atlamak* için kullanılan özel bir sensördür. Piksel verisi *üretmez*. Bunun yerine, görüş alanı (frustum) içindeki modellerin bir *listesini* ve bu modellerin kamera çerçevesine göre *pozlarını* (pose) raporlar. Görev planlama ("kutuyu bul ve al") gibi algoritmaları test etmek için kullanılır.

5. Programatik Kontrol: C++ Eklenti (Plugin) Sistemi

Mimari Konsept: Gazebo'nun *en* güçlü özelliği, çekirdek kodunu değiştirmeden simülasyonun hemen her yönünü kontrol etmeye ve genişletmeye olanak tanıyan C++ eklenti (plugin) sistemidir. Eklentiler, simülasyon dünyasına, modellere, sensörlere veya arayüze bağlanan, dinamik olarak yüklenen paylaşımlı kütüphanelerdir (`.so` dosyaları).

Her eklenti tipi, SDF/URDF içinde belirli bir etiketin altına yerleştirilir:

XML Yapılandırması (Dünya Eklentisi - `<world>` içine):

- **Amaç:** Tüm dünyayı kontrol eder. Modelleri programatik olarak ekleyip çıkarabilir (fabrika işlevi), yerçekimini değiştirebilir.

```

<world name="default">
  ...
  <plugin name="my_world_plugin" filename="libmy_world_plugin.so">
    <parameter_1>deger_1</parameter_1>
  </plugin>
</world>

```


XML Yapılandırması (Model Eklentisi - `<model>` içine):

- **Amaç:** Belirli bir modele bağlanır. O modelin linklerine kuvvet/tork uygulayabilir, eklemlerini kontrol edebilir (örn. `model_push` örneği).

```
<model name="my_robot">
  ...
  <plugin name="my_model_plugin" filename="libmy_model_plugin.so">
    <target_link>chassis_link</target_link>
    <force>10.0</force>
  </plugin>
</model>
```

XML Yapılandırması (Sensör Eklentisi - `<sensor>` içine):

- **Amaç:** Belirli bir sensöre bağlanır. Ham sensör verisine erişir, onu işler veya manipüle eder (örn. `libgazebo_ros_camera.so` bir sensör eklentisidir).

```
<sensor name="my_sensor" type="ray">
  ...
  <plugin name="my_sensor_plugin" filename="libmy_sensor_plugin.so">
    <ros_topic_name>/my_custom_laser</ros_topic_name>
  </plugin>
</sensor>
```

XML Yapılandırması (GUI Eklentisi - `<gui>` içine):

- **Amaç:** `gzclient` 'in (istemci) arayüzünü genişletir. Render penceresinin üzerine özel QT widget'ları (butonlar, kaydırıcılar) ekler.

```
<gui>
  <plugin name="my_gui_plugin" filename="libmy_gui_plugin.so"/>
</gui>
```

6. ROS Entegrasyonu (Genişletilmiş): Köprüler ve Kontrol

Mimari Konsept: Gazebo'nun ROS ile entegrasyonu, `gazebo_ros_pkgs` adı verilen ve Bölüm 5'te açıklanan **Eklenti (Plugin) Sistemi**ni kullanan bir dizi özel eklenti aracılığıyla sağlanır. Bu eklentiler, Gazebo'nun dahili **Gazebo Transport** (Protobuf) mesajlarını standart **ROS Transport** (ROS msgs) mesajlarına ve servislerine "köprüler".

6.1 URDF'nin Sınırları ve `<gazebo>` Etiketi

Sorun: URDF dosyaları, Gazebo'nun ihtiyaç duyduğu `<sensor>`, `<plugin>` veya `<friction>` gibi SDF'ye özgü etiketleri anlayamaz.

Çözüm: `<gazebo>` etiketi, bir URDF dosyası içinde bir "XML kaçış ambarı" veya "doğrudan geçiş" mekanizması olarak işlev görür. Bir URDF ayrıştırıcısı (örn. ROS'taki `robot_state_publisher`) bu etiketi görmezden gelir, ancak Gazebo'nun URDF'den-SDF'ye dönüştürücüsü bu etiketin *içeriğini* okur ve

doğrudan oluşturulan SDF modeline kopyalar.

XML Yapılandırması (URDF içine Sensör + ROS Eklentisi Ekleme):

```
<!--  
  Bu bir .urdf veya .xacro dosyasıdır.  
  URDF'de "camera_link" adında bir <link> tanımlanmış olmalıdır.  
-->  
<gazebo reference="camera_link">  
  <!--  
    Bu blok, Gazebo'nun SDF dönüştürücüsü tarafından okunacak  
    ve "camera_link"e eklenecektir.  
  -->  
  <sensor type="camera" name="camera1">  
    <update_rate>30.0</update_rate>  
    <camera name="head">  
      <horizontal_fov>1.396</horizontal_fov>  
      <image><width>800</width><height>800</height></image>  
      <clip><near>0.02</near><far>300</far></clip>  
    </camera>  
  
    <!--  
      Sensör Eklentisi: Bu C++ eklentisi (libgazebo_ros_camera.so),  
      ebeveyn <sensor> ögesinden simüle edilmiş verileri alır  
      ve bu verileri ROS Konuları (imageTopicName) aracılığıyla yayınlar.  
      Bu, simülasyon (SDF) ile ROS (eklenti) arasındaki en sıkı entegrasyon noktasıdır.  
    -->  
    <plugin name="camera_controller" filename="libgazebo_ros_camera.so">  
      <alwaysOn>true</alwaysOn>  
      <cameraName>rrbot/camera1</cameraName>  
      <imageTopicName>image_raw</imageTopicName>  
      <cameraInfoTopicName>camera_info</cameraInfoTopicName>  
      <frameName>camera_link</frameName>  
    </plugin>  
  </sensor>  
</gazebo>
```

6.2 ROS Aktüatör Yönetimi: `ros_control`

Mimari Konsept: Robot motorlarını ROS üzerinden kontrol etmenin standart yolu `ros_control` paketidir.

Bu sistem, üç parçalı bir yapılandırma zincirine dayanır:

1. `<transmission>` (URDF): Soyut bir URDF `<joint>` 'ini, ROS kontrolörlerinin komut gönderebileceği somut bir "donanım arayüzüne" (örn. `EffortJointInterface`) bağlar.
2. `<plugin>` (URDF içinde `<gazebo>` etiketiyle): `libgazebo_ros_control.so` eklentisini yükler. Bu eklenti, tüm `<transmission>` etiketlerini okur ve Gazebo'nun fizik motoru ile ROS'un Kontrolör Yöneticisi (ControllerManager) arasında köprü kurar.
3. **YAML Yapılandırma Dosyası (XML Değil):** Asıl kontrolörün (örn. `joint_position_controller`) ve onun PID kazançlarının (`p` , `i` , `d`) yüklendiği yerdir. Bu dosya, `roslaunch` tarafından ROS Parametre Sunucusuna yüklenir.

XML Yapılandırması (URDF içinde):

```
<!-- 1. ADIM: Aktüatör Arayüzünü Tanımlama -->  
<transmission name="left_wheel_trans">  
  <type>transmission_interface/SimpleTransmission</type>
```

```

<!-- Bu 'name', URDF'deki bir <joint> ile eşleşmelidir -->
<joint name="left_wheel_joint">
  <!--
    Kontrol tipini belirtir: Efor/Tork, Hız veya Pozisyon.
    Bu, YAML dosyasındaki kontrolör tipiyle uyumlu olmalıdır.
  -->
  <hardwareInterface>hardware_interface/EffortJointInterface</hardwareInterface>
</joint>

<actuator name="left_wheel_motor">
  <hardwareInterface>hardware_interface/EffortJointInterface</hardwareInterface>
  <mechanicalReduction>1</mechanicalReduction>
</actuator>
</transmission>
<!-- Sağ tekerlek için benzer bir <transmission> bloğu... -->

<!-- 2. ADIM: ros_control Eklentisini Yükleme -->
<!-- Bu etiket genellikle URDF dosyasının köküne yakın bir yere yerleştirilir -->
<gazebo>
  <plugin name="gazebo_ros_control" filename="libgazebo_ros_control.so">
    <!--
      Tüm ROS topic/servisleri için bir isim alanı.
      Birden fazla robot için zorunludur.
    -->
    <robotNamespace>/my_robot</robotNamespace>
    <controlPeriod>0.001</controlPeriod>
  </plugin>
</gazebo>

```

6.3 ROS Servis Arayüzleri

Mimari Konsept: `gazebo_ros_pkgs` (özellikle `gazebo_ros_api_plugin`), simülasyonu dışarıdan (örn. bir komut satırı veya ROS düğümü) yönetmek için bir dizi standart ROS Servisi sağlar.

- `/gazebo/spawn_urdf_model` veya `/gazebo/spawn_sdf_model` : Simülasyona programatik olarak model ekler.
- `/gazebo/delete_model` : Bir modeli simülasyondan siler.
- `/gazebo/get_model_state` : Bir modelin pozunu ve hızını sorgular.
- `/gazebo/set_model_state` : Bir modelin pozunu ve hızını (teleport) ayarlar.
- `/gazebo/apply_body_wrench` : Bir linke belirli bir süre boyunca kuvvet/tork uygular.
- `/gazebo/apply_joint_effort` : Bir ekleme belirli bir süre boyunca tork uygular.
- `/gazebo/pause_physics` ve `/gazebo/unpause_physics` : Simülasyonu duraklatır ve devam ettirir.

7. Gelişmiş Varlıklar: Aktörler `<actor>`

Mimari Konsept: Simülasyonda "canlı" bir dünya (örn. yürüyen insanlar, gezinen arabalar) yaratmak, eğer her biri tam fiziksel model olsaydı, muazzam bir hesaplama yükü getirirdi. "Aktörler" (`<actor>`) bu sorunu çözer. Bir aktör:

- Fiziğe tabi değildir (çarpışmalardan veya yerçekiminden etkilenmez).
- Önceden tanımlanmış bir yörüngeyi (`<script>`) takip eder.
- Bir iskelet animasyonunu (`<animation>`) oynatır.
- Render motoru tarafından görselleştirilir ve sensörler (Kamera, Lidar) tarafından algılanabilir.

XML Yapılandırması (`.world` dosyası içinde):

```
<actor name="actor_1">
  <!-- Görünüm (3D model - .dae veya .bvh dosyası) -->
  <skin>
    <filename>walk.dae</filename>
  </skin>

  <!-- Oynatılacak iskelet animasyonu (yerinde yürüme döngüsü) -->
  <animation name="walking">
    <filename>walk.dae</filename>
    <!-- Animasyon hızını yürüme hızıyla eşleştir -->
    <interpolate_x>true</interpolate_x>
  </animation>

  <!-- Dünyadaki hareket yolu -->
  <script>
    <loop>true</loop> <!-- Yörüngeyi sonsuz döngüye al -->
    <trajectory id="0" type="walking"> <!-- type, <animation> name ile eşleşir -->
      <!-- Anahtar kareler: (zaman, poz) -->
      <waypoint>
        <time>0</time>
        <pose>0 2 0 0 0 -1.57</pose>
      </waypoint>
      <waypoint>
        <time>7.5</time>
        <pose>0 -2 0 0 0 -1.57</pose>
      </waypoint>
    </trajectory>
  </script>
</actor>
```

8. Sentez Sonucu

Bu genişletilmiş analiz, Gazebo'nun gücünün sadece mimarisinden veya sadece XML esnekliğinden gelmediğini; bu iki katmanın kesişiminden geldiğini göstermektedir.

- `ros_control` gibi karmaşık bir sistem, ancak `libgazebo_ros_control.so` (Mimari - Eklenti) ve `<transmission>` (Yapılandırma - XML) bir araya geldiğinde çalışır.
- Gerçekçi bir IMU simülasyonu, `gzserver` 'daki gürültü modelleri (Mimari - Fizik) ve bu modelleri ayarlayan `<noise>` ve `<bias_mean>` etiketleri (Yapılandırma - XML) ile mümkün olur.
- URDF'nin (ROS standardı) Gazebo'da (SDF standardı) çalışabilmesi, `<gazebo>` etiketi (Yapılandırma - XML) ve bu etiketi yorumlayan `gazebo_ros_pkgs` (Mimari - Eklenti) sayesinde mümkündür.